

Apunte_Maestro_Parcial1_TDI

Apunte Maestro — Parcial 1 de Teoría de la Información

Refinado a partir de: apunte "PARCIAL-1 Apunte Completo", apuntes de cátedra (Fabricio Rubio), Área Temática 1 — Codificación de Fuente, y la planilla de cátedra "Compresor PPM-C". Cubre unidades 1 a 5. Todos los cálculos numéricos de la sección del ejemplo integrador fueron recalculados y verificados con precisión exacta (fracciones), no a mano, para eliminar errores de arrastre.

Cómo usar este documento

El apunte tiene dos capas que conviene leer juntas:

- Teoría por unidad** (secciones 1 a 5): definiciones, fórmulas y el razonamiento de por qué cada fórmula es como es. Sirve para responder la parte teórica y para entender ejercicios de canal, muestreo y Markov.
- Ejemplo integrador único** (sección 6): la palabra ABRACADABRA se usa como hilo conductor y se comprime con **los métodos** que entran en el parcial — código de bloque, Huffman estático, Shannon, Fano, codificación aritmética, BWT, Huffman dinámico, PPM-C, Lempel-Ziv (ventana deslizante) y Lempel-Ziv-Welch — mostrando en cada uno el procedimiento, no solo el resultado. Al final hay una tabla comparativa que responde la pregunta "¿cuál comprimió mejor y por qué?".

Se eligió ABRACADABRA porque es el mismo ejemplo que ya aparece en el material de cátedra para BWT y PPM-C, así que en vez de tener un ejemplo distinto por técnica (difícil de comparar), acá todos parten de la misma fuente y son directamente comparables entre sí.

0. Qué se evalúa en el Parcial 1 (mapa rápido)

El parcial se arma cruzando el examen modelo (4 variantes, temas 1–4), el parcial recuperatorio 2023, los apuntes de cátedra y el Área Temática 1. Estructura de las 4 variantes: siempre son **3 bloques** (fuente independiente, fuente de Markov, canal), cambiando el método de codificación exigido en el bloque A:

Variante	Método de codificación (bloque A)	Ejercicio de Markov	Canal BSC
Tema 1	Shannon	p(eeii)	C con $p(0 0)=0,95$
Tema 2	Huffman dinámico + Sibling Property	p(eiua)	C con $p(0 0)=0,5$
Tema 3	Aritmética (3 decimales)	p(uuaa)	I con $p(0 0)=0,85$, $p(0)=0,7$
Tema 4	LZ ventana deslizante (8 caract.)	p(aaii)	I con $p(0 0)=0,85$, $p(1)=0,7$

El Recuperatorio Tema 2 (2023) agrega dos elementos que no están en el modelo: un **canal binario asimétrico** (no BSC, hay que usar el método general $I(A; B) = H(B) - H(B|A)$, nunca $1 - H(p)$) y una **decodificación de Huffman dinámico** bit a bit.

Lista cerrada de preguntas teóricas (tal como se repiten en los parciales corregidos):

- Orden óptimo LZW + Huffman → primero LZW (saca redundancia de largo alcance), después Huffman (códigos cortos a lo frecuente).
- Fuente de 7 símbolos con $p_i = r^{-l_i}$ para que Huffman=Shannon=Fano y Kraft=1, H=L.
- Necesidad de la compansión → eficiencia de almacenamiento/transmisión y ahorro de costos.
- Las 5 propiedades de I : positiva, inversa a p , crece con la cantidad de sucesos, aditiva, H máxima con equiprobabilidad.
- PCM vs. DPCM: PCM cuantifica cada muestra; DPCM cuantifica la diferencia entre muestras (menor rango → menos bits, más robusto a ruido).
- Peor canal uniforme ternario → todas las probabilidades en 1/3, $C = 0$; emisor y receptor se consideran independientes.
- Métodos estadísticos estáticos: de doble lectura (cuentan y después codifican) y sin pérdida.
- A mayor redundancia, mayor compactación posible.
- Todo código instantáneo es unívoco; Huffman siempre produce un código instantáneo (prefijo).
- Los árboles de Huffman estático y dinámico **no** tienen por qué coincidir.
- Peor BSC: $p = 0,5 \Rightarrow C = 0$.
- Los archivos comprimidos tienen mayor entropía y menor redundancia (se eliminó lo predecible).
- BWT inversa: reconstruir la cadena original a partir de la transformada.

1. Unidad 1 — Información y entropía

1.1 Medida de información

$$I_i = \log_2 \left(\frac{1}{p_i} \right) = -\log_2 p_i \quad [\text{bits}]$$

Un bit es la información de especificar 1 de 2 alternativas igualmente probables. Un suceso con $p = 1$ aporta $I = 0$ (no había incertidumbre que resolver).

Las 5 propiedades (pregunta teórica fija):

1. $I_i \geq 0$ siempre (se anula solo con $p_i = 1$).
2. Inversamente proporcional a p_i : lo raro informa mucho, lo seguro informa poco.
3. Crece con la cantidad de sucesos posibles.
4. Aditiva para sucesos independientes: $I(E_1 \cap E_2) = I(E_1) + I(E_2)$.
5. La entropía (información promedio) se maximiza con sucesos equiprobables.

1.2 Entropía (Shannon)

$$H(A) = \sum_{i=1}^n p_i I_i = - \sum_{i=1}^n p_i \log_2 p_i \quad [\text{bits/símbolo}]$$

Acotación: $0 \leq H(A) \leq \log_2 n$. El mínimo ($H = 0$) es una fuente determinística (un símbolo con $p = 1$); el máximo es equiprobabilidad total.

1.3 Redundancia y rendimiento

$$H_{max} = \log_2 q, \quad \eta = \frac{H(A)}{H_{max}}, \quad R = 1 - \eta$$

Interpretación: R mide qué fracción del "espacio" de símbolos se usa de más allá de lo estrictamente necesario para transportar la información que realmente hay.

(El cálculo completo de H , η , R sobre un ejemplo real se hace en la sección 6.1, con la propia palabra ABRACADABRA — mismo procedimiento que el ejemplo clásico de vocales del apunte de cátedra.)

2. Unidad 2 — Canal de información

2.1 El canal como matriz de transición

$$P(B|A) = \begin{bmatrix} P(b_1|a_1) & \cdots & P(b_s|a_1) \\ \vdots & \ddots & \vdots \\ P(b_1|a_q) & \cdots & P(b_s|a_q) \end{bmatrix}$$

Cada fila suma 1 (es una distribución de probabilidad condicionada a la entrada a_i).

2.2 Relaciones probabilísticas

$$P(b_j) = \sum_i P(a_i)P(b_j|a_i) \quad (\text{prob. total})$$

$$P(a_i|b_j) = \frac{P(a_i)P(b_j|a_i)}{P(b_j)} \quad (\text{Bayes})$$

2.3 Canal uniforme, simétrico, BSC

- **Uniforme**: todas las filas son la misma permutación.
- **Simétrico**: subconjunto del uniforme, con simetría también en columnas (el ruido afecta igual a cualquier símbolo).
- **BSC** (Canal Binario Simétrico): el caso particular más estudiado, matriz $\begin{bmatrix} 1-p & p \\ p & 1-p \end{bmatrix}$.

2.4 Entropías del sistema e información mutua

$$H(A, B) = H(A) + H(B|A) = H(B) + H(A|B)$$

$$I(A; B) = H(A) + H(B) - H(A, B) = H(A) - H(A|B) = H(B) - H(B|A)$$

- $H(A)$: incertidumbre a priori sobre la entrada.
- $H(B|A)$: **ruido** (lo que el canal agrega, información inútil).
- $H(A|B)$: **equivocación** (lo que sigue sin saberse del símbolo original después de leer la salida).
- $I(A; B) \geq 0$ siempre; $I = 0 \iff A$ y B independientes; es simétrica.

2.5 Capacidad de canal

$$C = \max_{P(a)} I(A; B) \quad [\text{bits/símbolo}]$$

$$C_{BSC} = 1 - H(p) = 1 + p \log_2 p + (1 - p) \log_2 (1 - p)$$

- $p = 0$ o $p = 1$: canal perfecto (con o sin inversión), $C = 1$.
- $p = 0,5$: canal inútil, $C = 0$ (nunca se sabe si un 1 recibido vino de un 0 o de un 1 enviado).

Canal asimétrico (no BSC) — lo que hay que recordar del Recuperatorio 2023: si las filas de la matriz no son todas la misma permutación, **no vale** $C = 1 - H(p)$. Hay que ir al método general:

1. Fijar $P(a_i)$.
2. Calcular $P(b_j) = \sum_i P(a_i)P(b_j|a_i)$.
3. Calcular $H(A)$, $H(B)$, y el ruido $H(B|A) = \sum_i P(a_i)H(\text{fila } i)$.
4. $I(A; B) = H(B) - H(B|A)$.
5. $C = \max_{P(a)} I(A; B)$ (no hay fórmula cerrada; se optimiza numéricamente, o se prueba con $P(a)$ equiprobable como primera aproximación).

2.6 Ejemplos resueltos (verificados)

BSC con $p(0|0) = 0,95 \rightarrow p = 0,05$: $H(0,05) \approx 0,2864$, $C = 1 - 0,2864 = \mathbf{0,714}$ bits/símbolo.

BSC con $p(0|0) = 0,5 \rightarrow p = 0,5$: $C = 0$ (canal inútil).

BSC con $p(0|0) = 0,85$, $P(0) = 0,7$ (información mutua, no capacidad): $p = 0,15$. $P(0)_{sal} = 0,64$, $H(A) = 0,8813$, $H(B) = 0,9427$, $H(B|A) = 0,6098 \rightarrow I(A; B) = 0,8813 + 0,9427 - 1,4911 \approx \mathbf{0,333}$ bits.

Canal asimétrico del Recuperatorio Tema 2: $P(0|0) = 0$, $P(1|0) = 1$ (la fila $a_1=0$ determina la salida); $P(0|1) = 1/8$, $P(1|1) = 7/8$. Con entrada equiprobable: $P(0)_{sal} = 0,0625$, $P(1)_{sal} = 0,9375$, $H(B) = 0,3373$, $H(B|A) = 0,5 \cdot 0 + 0,5 \cdot 0,5436 = 0,2718 \rightarrow I(A; B) = 0,0655$ bits. (La capacidad real se logra con $P(0) \approx 0,626$, dando $C \approx 0,0691$.)

⚠ Una resolución manuscrita de 2023 dio $I(A; B) = -0,115$ bits para este mismo canal. Es matemáticamente imposible: $I(A; B) \geq 0$ siempre. Si en un cálculo propio da negativo, hay un error de signo o de fórmula — revisar antes de entregar.

3. Unidad 3 — Muestreo, cuantificación y codificación de la muestra

3.1 Muestreo (Nyquist)

$$f_s \geq 2B \quad (\text{frecuencia de Nyquist } f_N = 2B)$$

Si $f_s < 2B$ aparece **aliasing** (frecuencias falsas, irreconstruibles).

3.2 Cuantificación uniforme

$$\Delta = \frac{V_{max} - V_{min}}{L}, \quad L = 2^n, \quad -\frac{\Delta}{2} \leq e_q \leq \frac{\Delta}{2}$$

3.3 Tasa de bits

$$R = f_s \cdot n \cdot c$$

Caso	f_s	n	c	R
Telefonía	8 kHz	8	1	64 kbps
CD	44,1 kHz	16	2	≈1411 kbps

3.4 PCM vs. DPCM vs. ADPCM vs. modulación delta

- **PCM:** cuantifica y codifica cada muestra directamente.
- **DPCM:** codifica la *diferencia* entre muestras sucesivas. Como esa diferencia tiene rango dinámico menor, se necesitan menos bits para el mismo error. Se envía código PCM completo periódicamente para evitar que un error se propague indefinidamente.
- **ADPCM:** usa un predictor adaptativo (emisor y receptor comparten el mismo) — la diferencia predicción-muestra tiene rango aún menor.
- **Modulación delta:** DPCM de 1 bit (la señal solo sube o baja).

Ejemplo resuelto (con tolerancia): señal $[-10, 10]$ V, $\Delta = 2 \mu V$. Error de cuantización $e_q = \Delta/2 = 1 \mu V$. PCM: $L = 20V/2\mu V = 10^7$ niveles $\rightarrow n = \lceil \log_2 10^7 \rceil = \mathbf{24}$ bits/muestra. DPCM (rango entre muestras 2V, mismo Δ): $L = 10^6 \rightarrow n = \mathbf{20}$ bits/muestra. Con $f_{max} = 2\text{kHz}$

($f_s = 4\text{kHz}$) y 5 min de audio: PCM = 96 kbps ($2,88 \times 10^7$ bits); DPCM = 80 kbps ($2,4 \times 10^7$ bits) → **DPCM ahorra ≈17%** manteniendo el mismo error.

4. Unidad 4 — Códigos de fuente: teoría

4.1 Clasificación de códigos

- **Bloque** (longitud fija, ej. ASCII) vs. **no bloque** (longitud variable, corto para lo frecuente).
- **Singular** (dos símbolos comparten palabra código → inútil) vs. **no singular**.
- **Unívoco**: cada secuencia codificada tiene una única descomposición posible.
- **Instantáneo / prefijo**: subconjunto de los unívocos; se puede decodificar apenas se completa cada palabra, porque ninguna es prefijo de otra. Todo instantáneo es unívoco (no al revés).

4.2 Desigualdad de Kraft

$$\sum_{i=1}^q r^{-l_i} \leq 1$$

- = 1: instantáneo, árbol de prefijos lleno, y ahí $H = L$ (código compacto).
- < 1: instantáneo, sobra espacio en el árbol.
- > 1: no existe código instantáneo con esas longitudes.

McMillan es la misma desigualdad pero como condición *necesaria* (no suficiente) para códigos unívocos — puede cumplirse Kraft con un código que no sea instantáneo pero sí unívoco.

4.3 Longitud promedio, eficiencia, Primer Teorema de Shannon

$$L = \sum_i p_i l_i, \quad \eta = \frac{H(X)}{L}, \quad R = 1 - \eta$$

$$H(X) \leq L < H(X) + 1$$

Un código es **compacto** (óptimo, $\eta = 100\%$) cuando $p_i = r^{-l_i}$ para todo i (longitudes enteras exactas).

4.4 Los tres métodos estáticos — el procedimiento, en abstracto

Huffman (óptimo — minimiza L):

1. Ordenar probabilidades de mayor a menor.
2. Combinar los r símbolos menos probables en un nodo cuya probabilidad es la suma.
3. Reordenar y repetir hasta que quede un único nodo (raíz, prob=1).
4. Asignar 0/1 a cada rama y leer el código de raíz a hoja.

Shannon: $l_i = \lceil \log_2(1/p_i) \rceil$. Se calcula la función acumulada (FA) de cada símbolo, se pasa a binario, y se toman los primeros l_i dígitos.

Fano: ordenar por probabilidad decreciente, dividir recursivamente en dos grupos cuya suma de probabilidades sea lo más parecida posible a 0,5 cada uno, asignando 0 a un grupo y 1 al otro, y repitiendo dentro de cada grupo.

Relación entre los tres:

$$L_{Huffman} \leq L_{Fano} \leq L_{Shannon}$$

Huffman nunca es peor que los otros dos; la igualdad ocurre cuando $p_i = r^{-l_i}$ exactamente (fuente "compatible" con potencias de 2).

4.5 Extensión de orden n

Agrupar símbolos en bloques de n reduce el desperdicio por redondeo (que en el peor caso es 1 bit *por símbolo codificado*, no por bloque):

$$H(S^n) = n \cdot H(S), \quad H(S) \leq \frac{L_n}{n} < H(S) + \frac{1}{n}, \quad \lim_{n \rightarrow \infty} \frac{L_n}{n} = H(S)$$

5. Unidad 5 — Fuentes dependientes y códigos avanzados: teoría

5.1 Fuente de Markov

Orden m : $P(X|X_{m-1}, \dots, X_0)$. Estados = últimas m emisiones, hay q^m estados posibles. Matriz de transición: cada fila suma 1, $\sum_j P(S_j|S_i) = 1$.

5.2 Distribución estacionaria y ergodicidad

Un proceso es **ergódico** si, a largo plazo, el promedio temporal (una trayectoria larga) coincide con el promedio de conjunto (muchas realizaciones). La distribución estacionaria π cumple:

$$\pi P = \pi, \quad \sum_i \pi_i = 1$$

Tres formas de hallarla: (1) iterativa — elevar P a una potencia grande hasta que converja; (2) analítica — resolver el sistema de ecuaciones + normalizar; (3) álgebra lineal — autovector de autovalor 1, normalizado.

5.3 Codificación eficiente de Markov (un código por estado)

Se construye un Huffman distinto para las transiciones desde cada estado. El primer símbolo se codifica con bloque fijo (define el estado inicial), y desde ahí se usa el diccionario del estado actual.

$$L_M(S_i) = \sum_j l(S_j)P(S_j|S_i), \quad L_M = \sum_i P(S_i)L_M(S_i), \quad H(S_M) \leq H(S)$$

Conocer el estado previo reduce la incertidumbre, por eso $H(S_M)$ (entropía de Markov) es menor o igual que la entropía de memoria nula $H(S)$ — y por eso L_M puede quedar por debajo de $H(S)$. Desventaja: hay que transmitir/conocer todos los diccionarios (uno por estado).

5.4 Ejemplo resuelto (de cátedra) — secuencia de vocales AEEEEEEOOIIAAOOOEEEA

Con la matriz de transición dada, $H(S) = 1,8149$ bits/símbolo (memoria nula) pero $H(S_M) \approx 1,186 \leq H(S)$, $L_M \approx 1,299$ bits/símbolo (contra $L = 2$ de código bloque), y $\pi = (0,20, 0,45, 0,10, 0,25)$ para (A,E,I,O). Aprovechar la memoria de la fuente baja la longitud promedio en $\approx 35\%$ respecto del código de bloque.

5.5 Ergodicidad — qué la rompe

No-ergodicidad: dependencia fuerte del estado inicial, estados inaccesibles desde otros (el sistema queda "atrapado"), y promedio temporal $\neq$ promedio estadístico.

6. Ejemplo integrador: comprimiendo ABRACADABRA con los siete métodos

Se usa siempre la misma fuente para poder comparar resultados en igualdad de condiciones. ABRACADABRA tiene **11 símbolos**, alfabeto $\{A, B, C, D, R\}$ ($q = 5$).

6.1 Punto de partida: frecuencias y entropía

Símbolo	Frecuencia	p_i	$I_i = -\log_2 p_i$
A	5	5/11 = 0,4545	1,1375 bits
B	2	2/11 = 0,1818	2,4594 bits
R	2	2/11 = 0,1818	2,4594 bits
C	1	1/11 = 0,0909	3,4594 bits
D	1	1/11 = 0,0909	3,4594 bits

$$H(S) = \sum p_i I_i = \mathbf{2,0404} \text{ bits/símbolo}$$

$$H_{max} = \log_2 5 = 2,3219, \quad \eta = \frac{2,0404}{2,3219} = \mathbf{87,87\%}, \quad R = \mathbf{12,13\%}$$

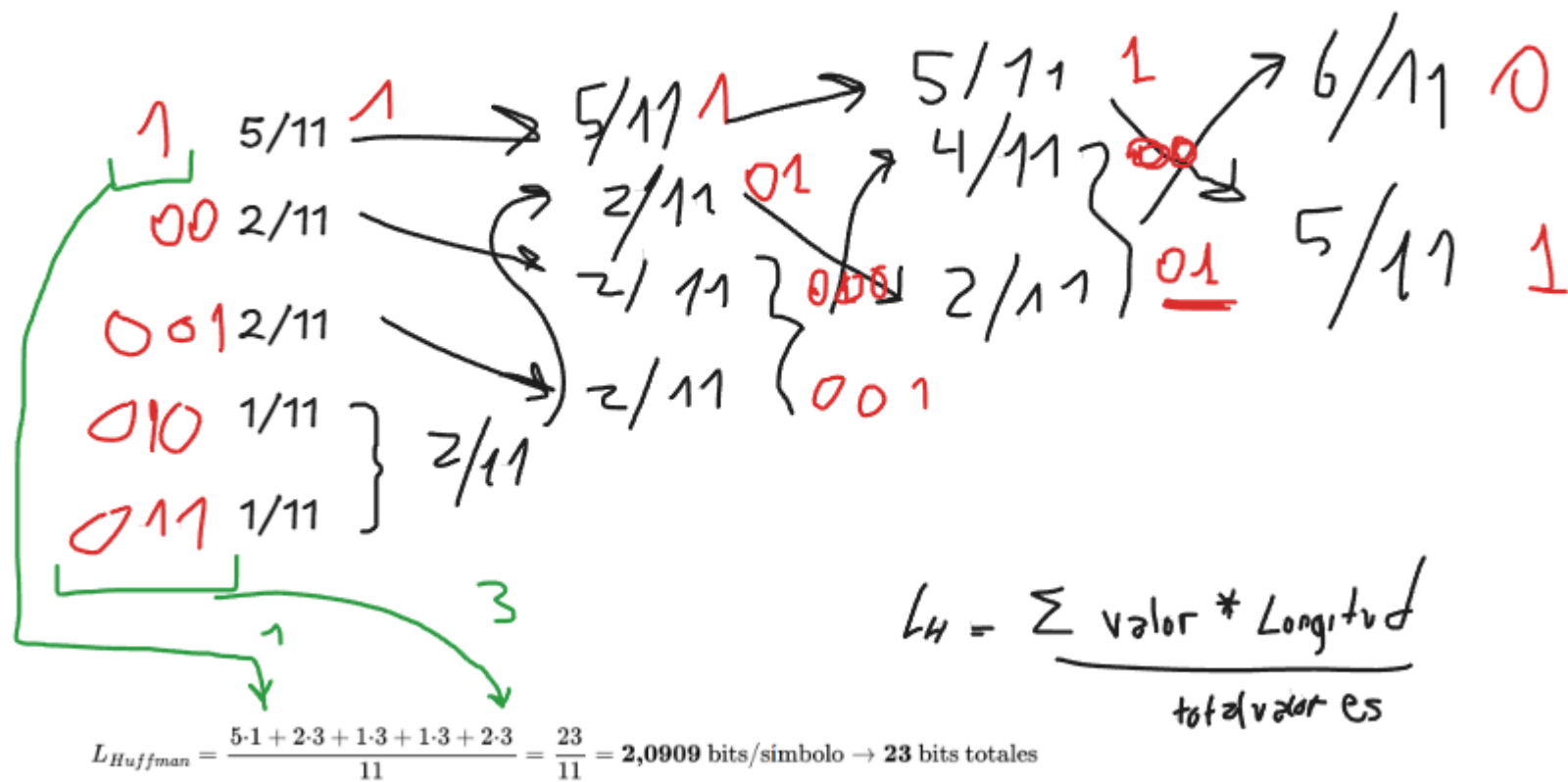
Cota teórica absoluta para comprimir esta secuencia completa: $H(S) \times 11 = \mathbf{22,44}$ bits. Ningún método sin pérdida puede bajar de ahí de manera sostenida — es la vara con la que se mide todo lo que sigue.

Código de bloque (longitud fija), como referencia de "no comprimir": $q = 5 \Rightarrow l = \lceil \log_2 5 \rceil = 3$ bits/símbolo $\rightarrow$ **33 bits** totales, $\eta = H/3 = 68,01\%$.

6.2 Huffman estático

Se ordenan los símbolos de mayor a menor probabilidad — **A, B, R, C, D** — y en cada paso se combinan los dos símbolos menos probables (C y D primero, ambos con $p = 1/11$) en un nodo cuya probabilidad es la suma, reordenando y repitiendo hasta llegar a un único nodo raíz.

HUFFMAN ESTATICO



$$L_H = \frac{\sum \text{valor} * \text{Longitud}}{\text{total valores}}$$

Símbolo	p_i	Código	l_i
A	5/11	0	1
B	2/11	111	3
R	2/11	110	3
C	1/11	101	3
D	1/11	100	3

$$L_{Huffman} = \frac{5 \cdot 1 + 2 \cdot 3 + 2 \cdot 3 + 1 \cdot 3 + 1 \cdot 3}{11} = \frac{23}{11} = 2,0909 \text{ bits/símbolo} \rightarrow 23 \text{ bits totales}$$

$$\eta = \frac{H}{L} = \frac{2,0404}{2,0909} = 97,58\%$$

Frente al código de bloque (33 bits), Huffman ahorra 10 bits (30% menos) sobre esta misma palabra.

6.3 Shannon

$l_i = \lceil \log_2(1/p_i) \rceil$, y el código sale de tomar los primeros l_i bits de la función acumulada (FA) en binario, ordenando los símbolos de mayor a menor probabilidad:

SHANNON

Longitud Promedio

$$\log_2 11/5 \leq A \leq \log_2 11/5 + 1 \rightarrow 2$$

$$\log_2 11/2 \leq B \leq \log_2 11/2 + 1 \rightarrow 3$$

$$\log_2 11/2 \leq R \leq \log_2 11/2 + 1 \rightarrow 3$$

$$\log_2 11/1 \leq C \leq \log_2 11/1 + 1 \rightarrow 4$$

$$\log_2 11/1 \leq D \leq \log_2 11/1 + 1 \rightarrow 4$$

Símbolo	p_i	FA	Dec	Bin	Long	Cod
A	5/11	0	0	0	2	0
B	2/11	5/11	0.45	0.011	3	011
R	2/11	7/11	0.64	0.101	3	101
C	1/11	9/11	0.81	0.1100	4	1100
D	1/11	10/11	0.90	0.1110	4	1110

$$L_{Shannon} = \frac{5 \cdot 2 + 2 \cdot 3 + 2 \cdot 3 + 1 \cdot 4 + 1 \cdot 4}{11} = \frac{30}{11} = 2,7273 \rightarrow 30 \text{ bits,}$$

$$L_{Shannon} = \frac{5 \cdot 2 + 2 \cdot 3 + 2 \cdot 3 + 1 \cdot 4 + 1 \cdot 4}{11} = \frac{30}{11} = 2,7273 \rightarrow 30 \text{ bits,} \quad \eta = 74,81\%$$

6.4 Fano

Se ordena por probabilidad decreciente — A, B, R, C, D — y se corta la lista en dos donde la suma de cada mitad se acerque más a 0,5, asignando 0 a un grupo y 1 al otro, y repitiendo recursivamente dentro de cada mitad hasta que cada grupo tenga un único símbolo.

FANO

0

5/11

0.45

0

10

2/11

0.18

10

0.54

1

110

2/11

0.18

110

0.36

11

1110

1/11

0.09

1110

0.09

111

1111

1/11

0.09

1111

0

5/11

0.45

0

100

2/11

100

0.54

1

101

2/11

101

10

110

1/11

110

11

111

1/11

111

1°

2°

3°

4°

2 formas de dividir

15 Sep

0

1

0

1

0

1

0

1

A

B

R

C

D

$$L_{Fano} = \frac{5 \cdot 1 + 2 \cdot 2 + 2 \cdot 3 + 1 \cdot 4 + 1 \cdot 4}{11} = \frac{23}{11} = 2,0909 \rightarrow 23 \text{ bits}$$

Símbolo	Código	l_i
A	0	1
B	10	2
R	110	3
C	1110	4
D	1111	4

$$L_{Fano} = \frac{5 \cdot 1 + 2 \cdot 2 + 2 \cdot 3 + 1 \cdot 4 + 1 \cdot 4}{11} = \frac{23}{11} = 2,0909 \rightarrow 23 \text{ bits}$$

Verificación de la relación teórica: $L_{Huffman}(2,0909) \leq L_{Fano}(2,0909) \leq L_{Shannon}(2,7273)$. Para esta fuente, Huffman y Fano **empatan exactamente** (ambos óptimos acá); Shannon queda claramente por detrás porque redondea las longitudes hacia arriba símbolo por símbolo sin margen de reacomodo.

6.5 Codificación aritmética

Se recorre el alfabeto una sola vez, en **el mismo orden decreciente de probabilidad usado en Shannon** (A, B, R, C, D — no alfabético), y se arma la tabla de intervalos acumulados en ese orden:

Símbolo	p_i	Intervalo $[Q, R)$
A	5/11	[0,0000 ; 0,4545)
B	2/11	[0,4545 ; 0,6364)
R	2/11	[0,6364 ; 0,8182)
C	1/11	[0,8182 ; 0,9091)
D	1/11	[0,9091 ; 1,0000)

Mecánica: en cada paso, $nuevo_low = low + rango \cdot Q_{símbolo}$, $nuevo_high = low + rango \cdot R_{símbolo}$, con $rango = high - low$. El intervalo se va achicando y "encerrando" cada vez más el número real que representa a toda la secuencia. Como el orden de los símbolos cambió respecto de una versión alfabética, **los intervalos de B, R, C y D también cambian** (no es solo cambiar el orden de la tabla: cada paso arrastra el nuevo $[Q, R)$ de cada símbolo).

Paso	Símbolo	Intervalo resultante
1	A	[0,00000000 ; 0,45454545)
2	B	[0,20661157 ; 0,28925620)
3	R	[0,25920361 ; 0,27422990)
4	A	[0,25920361 ; 0,26603374)
5	C	[0,26479190 ; 0,26541282)
6	A	[0,26479190 ; 0,26507414)
7	D	[0,26504848 ; 0,26507414)
8	A	[0,26504848 ; 0,26506014)
9	B	[0,26505378 ; 0,26505590)
10	R	[0,26505513 ; 0,26505551)
11	A	[0,26505513 ; 0,26505530)

Intervalo final exacto: [0,26505513, 0,26505530). Cualquier número dentro de ese rango identifica sin ambigüedad la secuencia completa; el más corto es `0,2650552` (7 dígitos decimales).

💡 Nota importante sobre "3 dígitos decimales": el enunciado del examen (Tema 3) pide 3 decimales, pero eso alcanza para secuencias **cortas** como `p(eeii)` (4 símbolos) — ahí el intervalo se cierra rápido. Para una palabra de 11 símbolos como esta, el intervalo final es tan angosto ($\approx 1,75 \times 10^{-7}$ de ancho) que hacen falta 7 decimales para distinguirlo; con solo 3 los intervalos de distintos mensajes se superpondrían. La forma correcta de leer la consigna "con 3 decimales" es: se **redondea recién al final**, cuando ya se tiene el intervalo definitivo, tomando el número más corto que cae adentro — no truncar en cada paso intermedio (eso rompe la precisión y el mensaje deja de ser decodificable).

Tamaño equivalente en bits: el ancho del intervalo final es $\approx 1,75 \times 10^{-7}$, y $-\log_2(1,75 \times 10^{-7}) \approx \mathbf{22,44}$ bits — **coincide casi exactamente con la cota de entropía** $H(S) \times 11 = 22,44$ bits calculada en 6.1. Esta es la gran ventaja teórica de la codificación aritmética: al no estar atada a longitudes de código enteras por símbolo (como sí les pasa a Huffman/Shannon/Fano), se acerca arbitrariamente al límite de Shannon. En la práctica, emitiendo el número con algún margen de seguridad, el mensaje ocupa **$\approx 23\text{-}24$ bits**.

6.6 Transformada de Burrows-Wheeler (BWT)

BWT no comprime por sí misma: reordena la cadena para agrupar caracteres repetidos, dejándola lista para que RLE o Huffman compriman mejor después. Procedimiento:

1. Agregar un símbolo de fin de cadena, `$` (menor que cualquier letra): `ABRACADABRA$` (12 caracteres).
2. Generar las 12 rotaciones cíclicas.
3. Ordenarlas lexicográficamente.
4. Tomar la última columna.

Rotación ordenada	Último carácter
\$ABRACADABRA	A
A\$ABRACADABR	R
ABRA\$ABRACAD	D
ABRACADABRA\$	\$
ACADABRA\$ABR	R
ADABRA\$ABRAC	C
BRA\$ABRACADA	A
BRACADABRA\$A	A
CADABRA\$ABRA	A
DABRA\$ABRACA	A
RA\$ABRACADAB	B
RACADABRA\$AB	B

Resultado: `ARD$RCAAAABB` (o, sin el `$`, simplificando: `RDARCAAAABB`).

Se nota inmediatamente el efecto buscado: la cadena original no tenía ninguna repetición consecutiva, pero la transformada tiene una corrida `AAAA` (4 caracteres seguidos) y una `BB`. Aplicando RLE a `RDARCAAAABB` se obtienen 7 "tokens" en vez de 11 símbolos sueltos: `R¹`

$D^1 \ A^1 \ R^1 \ C^1 \ A^4 \ B^2$ — la ganancia de RLE crece con el largo de las corridas, así que en textos más largos (donde BWT agrupa corridas mucho más largas) el ahorro es mucho mayor que en este ejemplo de 11 letras.

BWT inversa (lo que pide el parcial 2022 con la imagen manuscrita): se reconstruye la matriz completa ordenando y "pegando" columnas de forma iterativa (agregar la columna transformada a la izquierda de la tabla actual, reordenar, repetir n veces), hasta identificar la fila que termina en \$ — esa fila es la cadena original. Se verificó computacionalmente que la inversa de `ARD$RCAAAABB` reconstruye exactamente `ABRACADABRA$`.

6.7 Huffman dinámico (adaptativo)

A diferencia de Huffman estático, acá **no se conocen las frecuencias de antemano**: transmisor y receptor arrancan con un árbol de un único nodo, el **símbolo imaginario (NYT)**, y lo van construyendo símbolo a símbolo, en espejo, aplicando la Sibling Property para reubicar nodos cuando un incremento de peso lo exige. El desarrollo del árbol paso a paso (con sus intercambios) queda para la resolución gráfica propia; acá se deja directamente la tabla de resultados — código fijo usado para el NYT: $A=000$, $B=001$, $R=010$, $C=011$, $D=100$ ($q=5$):

Paso	Símbolo	¿Nuevo?	Bits emitidos	Long.
1	A	Sí	000	3
2	B	Sí	0001	4
3	R	Sí	00010	5
4	A	No	1	1
5	C	Sí	10011	5
6	A	No	01	2
7	D	Sí	000100	6
8	A	No	1	1
9	B	No	11	2
10	R	No	00	2
11	A	No	1	1

Stream total: 00000010001011001101000100111001 → **32** bits

Se ve claramente el patrón esperado: los primeros 4 símbolos nuevos (A, B, R, C, D — cada uno la primera vez que aparece) cuestan caro (3 a 6 bits, porque van con el camino al NYT + el código fijo completo), mientras que las repeticiones posteriores de A, B y R se emiten en 1 o 2 bits. Con solo 11 símbolos el "costo de aprendizaje" (transmitir el alfabeto la primera vez que aparece cada letra) todavía pesa mucho: el promedio final es $32/11 \approx 2,91$ bits/símbolo, peor que Huffman estático (2,09). Esto es normal y es justamente la desventaja conocida de los métodos adaptativos en mensajes cortos: **no hay tabla que transmitir de entrada, pero el aprendizaje sobre la marcha cuesta bits al principio**. Con un mensaje mucho más largo, el Huffman dinámico converge al mismo rendimiento que el estático (y lo supera en la práctica porque el estático necesita transmitir su tabla de códigos como cabecera).

⚠ El árbol exacto en cada paso (y por lo tanto los bits exactos) puede variar levemente según la convención de desempate que fije el enunciado cuando hay pesos iguales — el propio material de cátedra lo aclara para el ejercicio de decodificación del Recuperatorio 2023. Lo que **no** cambia es: (a) el mecanismo (NYT, sibling property, actualización en espejo) y (b) que el stream generado siempre se puede decodificar sin ambigüedad siguiendo exactamente el mismo algoritmo del lado receptor.

6.8 PPM-C (Prediction by Partial Matching)

Es el método más potente de los que entran en el parcial, y el que cierra la unidad 5. Tiene **dos etapas separadas**: primero un modelo estadístico por contexto (PPM-C en sí) decide probabilidades; después un codificador aritmético convierte esas probabilidades en bits.

Idea: para predecir el próximo símbolo, se mira el contexto (los n caracteres anteriores), empezando por el orden más alto disponible (acá, orden 2) y bajando de a uno si el símbolo no apareció nunca en ese contexto, hasta llegar a un orden -1 que es una distribución uniforme sobre un alfabeto base (en el ejemplo de cátedra, 64 símbolos → $1/64$ cada uno, representando un alfabeto genérico de texto, no solo las 5 letras de esta palabra).

El escape (ESC): cuando el contexto actual no conoció nunca el símbolo, se emite un evento especial ESC y se baja de orden. En PPM-C el "peso" del ESC no es fijo: $f(\text{ESC}) = U$, donde U es la cantidad de símbolos *distintos* ya vistos en ese contexto — cuantos más símbolos diferentes conoce un contexto, más probable se considera que aparezca uno todavía no visto.

$$P(\text{ESC}) = \frac{U}{total + U}, \quad P(s|\text{contexto}) = \frac{freq(s)}{total + U}$$

Etapa 1 — **modelado** (contextos de orden 2, 1, 0 y -1 para cada letra, actualizando tablas después de cada símbolo):

Paso	Símbolo	Búsqueda de contexto	Evento codificado
1	A	O2, O1, O0 vacíos → ESC·ESC·ESC	A en O–1 (1/64)
2	B	O2, O1 vacíos, O0 no tiene B → ESC·ESC·ESC(O0=1/2)	B en O–1 (1/64)
3	R	ídem, O0 no tiene R → ESC·ESC·ESC(O0=1/2)	R en O–1 (1/64)
4	A	O2, O1 vacíos, O0 sí tiene A → ESC·ESC	A en O0 (1/6)
5	C	O2(AC) vacío, O1(A) no tiene C, O0 no tiene C → ESC·ESC·ESC	C en O–1 (1/64)
6	A	O2, O1 vacíos, O0 sí tiene A → ESC·ESC	A en O0 (2/9)
7	D	O2(CA) vacío, O1(A) no tiene D, O0 no tiene D → ESC·ESC·ESC	D en O–1 (1/64)
8	A	O2, O1 vacíos (AD), O0 sí tiene A → ESC·ESC	A en O0 (1/4)
9	B	O2(DA) vacío, O1(A) sí tiene B → ESC	B en O1 (1/6)
10	R	O2(AB) sí tiene R → codifica directo	R en O2 (1/2)
11	A	O2(BR) sí tiene A → codifica directo	A en O2 (1/2)

El patrón clave: al principio todo son escapes hasta O–1 (nada de contexto conocido todavía); a mitad de camino el contexto de orden 0 ya predice bien las repeticiones de A; y hacia el final, con la secuencia "AB" y "BR" ya vistas, el orden 2 predice directamente R y A **sin ningún escape** — el modelo aprendió la estructura local de la palabra.

Eta­pa 2 — codificador aritmético: toma cada evento (ESC o símbolo) con su probabilidad y va angostando un único intervalo $[low, high)$, igual que en 6.5, pero encadenando varias probabilidades por símbolo (una por cada ESC que hubo que atravesar). Usa 3 reglas de renormalización para no perder precisión:

- **E1** ($high \leq 0,5$): todo el intervalo empieza con 0 → emitir 0, duplicar.
- **E2** ($low \geq 0,5$): todo el intervalo empieza con 1 → emitir 1, duplicar.
- **E3** ($0,25 \leq low < 0,5 < high \leq 0,75$): todavía ambiguo → no emitir, guardar un "bit pendiente" y seguir angostando; cuando después aparece E1 o E2, se resuelve emitiendo ese bit y luego los pendientes invertidos.

Bits emitidos por paso (contando los pendientes resueltos):

Paso	Símbolo	Bits en este paso
1	A	000000 (6)
2	B	0000001 (7)
3	R	0010001 (7)
4	A	10 (2)
5	C	00000001 (8)
6	A	— (2 bits pendientes)
7	D	01100101111 (11, incluye los 2 pendientes)
8	A	— (1 bit pendiente)
9	B	1001 (4)
10	R	— (esperando)
11	A	1 (1)
Flush	—	011 (3, cierre final)

Total: **49** bits

Comparación (misma base de 64 símbolos que usa el ejemplo): sin modelado, 11 símbolos × 6 bits (para indexar 1 de 64) = 66 bits. PPM-C logra **49 bits → 25,8% de reducción**, a pesar de que la palabra es corta y el modelo recién está aprendiendo.


 Estos 49 bits **no son comparables número contra número** con los 23-33 bits de Huffman/Shannon/Fano/aritmética de la sección anterior: aquellos métodos asumen un alfabeto cerrado y conocido de antemano de solo 5 símbolos, mientras que PPM-C usa un alfabeto base de 64 símbolos (más realista para texto real, donde no se sabe de antemano qué letras van a aparecer). La comparación justa de PPM-C es contra el código de bloque *de su propio alfabeto* (66 bits) — ahí sí se ve la ganancia real del modelado por contexto.

6.9 Lempel-Ziv (ventana deslizante, LZ77)

Es el método pedido en la variante Tema 4 del examen modelo ("LZ con ventana deslizante, ventana de 8 caracteres"). A diferencia de todo lo anterior, acá **no hay tabla de probabilidades**: el "diccionario" es directamente la parte del mensaje ya procesada (el *buffer de*

búsqueda), y en cada paso se busca ahí la coincidencia más larga con lo que sigue (el *buffer de mirar más allá*). Se emite una tupla (**desplazamiento, longitud, símbolo siguiente**).

Paso	Buffer de búsqueda (diccionario previo)	Buffer de mirar más allá (a procesar)	Coincidencia hacia atrás	Salida (marca generada)
1	(vacío)	ABRACADABRA	No hay, sig. A	(0,0,'A')
2	A	BRACADABRA	No hay, sig. B	(0,0,'B')
3	AB	RACADABRA	No hay, sig. R	(0,0,'R')
4	ABR	ACADABRA	Hay coinc. para "A". Está a 2 pos., long. 1. Sig. C	(2,1,'C')
5	ABRAC	ADABRA	Hay coinc. para "A". Está a 4 pos., long. 1	(4,1,'D')
6	ABRACAD	ABRA	Hay coinc. para "ABRA". Está a 6 pos., long. 4	(6,4,'eof')

Salida final: 6 tuplas en vez de 11 símbolos sueltos — los pasos 4, 5 y 6 ya no gastan una marca por letra, sino que "apuntan hacia atrás" y reaprovechan lo ya visto (el paso 6 reconstruye de una sola vez las 4 letras que faltan, ABRA). Si se acotan los campos a la ventana de 8 caracteres pedida (desplazamiento y longitud en 4 bits cada uno) más 3 bits para el símbolo (alfabeto de 5 letras + marca de fin), cada tupla ocupa ≈ 11 bits $\rightarrow 6 \times 11 \approx 66$ bits (cifra solo ilustrativa: el ancho exacto de cada campo lo fija el enunciado). Para un mensaje tan corto, el costo de emitir desplazamiento+longitud en cada tupla todavía no se paga solo; la ventaja de LZ77 aparece cuando el texto es lo bastante largo como para que las coincidencias sean mucho más largas que 1 o 4 caracteres.

6.10 Lempel-Ziv-Welch (LZW)

A diferencia de LZ77, LZW no emite desplazamiento+longitud: mantiene un **diccionario que arranca con el alfabeto base ya cargado** (acá, con códigos tipo ASCII: A=65, B=66, C=67, D=68, R=82) y le va agregando cada nueva secuencia de dos símbolos que encuentra, emitiendo siempre el índice de la cadena más larga ya conocida en el diccionario.

Paso (lectura)	Cadena acumulada (l)	Evaluación (l+x)	Salida (código emitido)	Nueva entrada al diccionario
1 (lee 'A')	A	Lee B, "AB" no está en el dicc.	65 (índice de A)	256 = AB
2 (lee 'B')	B	Lee R, "BR" no está	66 (índice de B)	257 = BR
3 (lee 'R')	R	Lee A, "RA" no está	82 (índice de R)	258 = RA
4 (lee 'A')	A	Lee C, "AC" no está	65 (índice de A)	259 = AC
5 (lee 'C')	C	Lee A, "CA" no está	67 (índice de C)	260 = CA
6 (lee 'A')	A	Lee D, "AD" no está	65 (índice de A)	261 = AD
7 (lee 'D')	D	Lee A, "DA" no está	68 (índice de D)	262 = DA
8 (lee 'A')	A	Lee B, "AB" sí está (256)	— (sigue acumulando)	—
9 (lee 'B')	AB	Lee R, "ABR" no está	256 (índice de AB)	263 = ABR
10 (lee 'R')	R	Lee A, "RA" sí está (258)	— (sigue acumulando)	—
11 (lee 'A')	RA	fin de cadena	258 (índice de RA)	EOF

Salida final: 9 códigos (65, 66, 82, 65, 67, 65, 68, 256, 258) en vez de 11 símbolos — el diccionario llegó a capturar "AB" y "RA" como unidades de 2 caracteres cada una (pasos 8 y 10, donde no se emite nada porque sigue "buscando algo más largo"). Con un ancho fijo de 9 bits por código (alcanza para llegar hasta el índice 263), el mensaje ocupa $9 \times 9 = 81$ bits: más que el propio código de bloque (33 bits), otra vez porque en un mensaje tan corto el diccionario recién está arrancando. La ganancia real de LZW (igual que la de LZ77) aparece en archivos largos, donde las entradas del diccionario terminan representando frases enteras que se repiten muchas veces.

7. Tabla comparativa final — ¿cuál comprimió mejor?

Grupo A — alfabeto cerrado y conocido de antemano (5 símbolos: A,B,C,D,R)

Método	Bits totales	Bits/símbolo	$\eta = H/L$	Comentario
Código de bloque	33	3,000	68,0%	Referencia, sin comprimir
Shannon	30	2,727	74,8%	Redondeo símbolo a símbolo, el más simple pero el menos eficiente de los estadísticos

Método	Bits totales	Bits/símbolo	$\eta = H/L$	Comentario
Huffman dinámico	32*	2,909*	70,1%*	*Incluye el costo de "aprender" el alfabeto; converge a Huffman estático con mensajes largos
Huffman estático	23	2,091	97,6%	Óptimo entre los códigos de longitud entera
Fano	23	2,091	97,6%	Empata con Huffman en esta fuente particular
Aritmética	≈23-24	≈2,1	≈93-96%	Prácticamente pegada a la cota teórica $H \cdot n = 22,44$ bits
Cota teórica ($H \times 11$)	22,44	2,040	100%	Límite inalcanzable en la práctica, pero se puede rozar

Grupo B — alfabeto general de 64 símbolos (más realista para texto)

Método	Bits totales	Reducción vs. bloque
Código de bloque (6 bits/símbolo)	66	—
PPM-C + aritmética	49	25,8%

Grupo C — métodos de diccionario (no usan tabla de probabilidades, sino coincidencias/índices)

Método	Unidades emitidas	Bits totales (ilustrativo)*	Comentario
LZ77 (ventana deslizante)	6 tuplas (desplaz., long., símbolo)	≈66	Empieza a "apuntar hacia atrás" recién en el paso 4
LZW	9 códigos de diccionario	≈81	Ya reutiliza "AB" y "RA" como unidades completas

*El bit-total de estos dos depende del ancho de campo que fije el enunciado (tamaño de ventana, tamaño de diccionario); son valores ilustrativos, no una cota cerrada como en Huffman/Shannon/Fano.

Conclusiones que hay que poder explicar en el parcial:

1. **Huffman $\approx$ Fano $\leq$ Aritmética $\leq$ cota de entropía**, siempre en ese orden o mejor (la aritmética nunca puede ser peor que Huffman en el límite, porque no está atada a longitudes enteras).
2. **Shannon es el más simple de calcular pero el menos eficiente** de los tres estadísticos clásicos — se nota en la tabla: 30 bits contra 23 de Huffman/Fano para la misma fuente.
3. Los métodos **adaptativos y de diccionario (Huffman dinámico, PPM-C, LZ77, LZW) pagan un costo de aprendizaje** al principio del mensaje — por eso en textos cortos como este pueden rendir peor en bits/símbolo que un Huffman estático calculado con las frecuencias ya conocidas. Su ventaja real aparece en mensajes largos, donde además **no hace falta transmitir ninguna tabla/cabecera** (el estático sí la necesita, lo cual en archivos chicos puede comerse toda la ganancia).
4. **BWT no compite en esta tabla** porque no es, por sí misma, un compresor: es un paso de *reordenamiento* que expone corridas repetidas para que RLE o Huffman las aprovechen después. Su beneficio crece con la longitud y la redundancia del texto, no se ve bien en una palabra de 11 letras.
5. **PPM-C es el método más potente en términos absolutos** (es el que efectivamente se usa en compresores reales de texto), pero para juzgarlo hay que compararlo contra la referencia de *su propio* alfabeto base, no contra un alfabeto reducido armado a medida de la palabra.
6. **LZ77 vs. LZW**: LZ77 mira hacia atrás en el propio texto ya leído (sin diccionario aparte, pero repite desplazamiento+longitud en cada tupla); LZW arma un diccionario explícito de frases que solo crece (nunca repite el mismo dato dos veces, pero necesita que ese diccionario tenga tiempo de poblarse). Por eso el orden recomendado en la pregunta teórica cerrada #1 (LZW antes que Huffman) tiene sentido: LZW saca la redundancia de "frases repetidas", y recién sobre ese resultado ya comprimido Huffman aprovecha lo que quede de asimetría en las frecuencias.

8. Chuleta de fórmulas

Concepto	Fórmula
Información	$I_i = -\log_2 p_i$
Entropía	$H = -\sum p_i \log_2 p_i$

Concepto	Fórmula
Entropía máxima	$H_{max} = \log_2 q$
Rendimiento / redundancia	$\eta = H/H_{max}, R = 1 - \eta$
Prob. de salida	$P(b_j) = \sum_i P(a_i)P(b_j a_i)$
Bayes (canal)	$P(a_i b_j) = \frac{P(a_i)P(b_j a_i)}{P(b_j)}$
Información mutua	$I(A; B) = H(A) - H(A B) = H(B) - H(B A)$
Capacidad	$C = \max_{P(a)} I(A; B)$
Capacidad BSC	$C_{BSC} = 1 - H(p)$
Canal asimétrico	$I(A; B) = H(B) - H(B A)$ — no usar $1 - H(p)$
Kraft	$\sum r^{-l_i} \leq 1$
Longitud promedio	$L = \sum p_i l_i$
Eficiencia de código	$\eta = H/L$
Primer Teorema de Shannon	$H \leq L < H + 1$
Shannon (longitudes)	$l_i = \lceil \log_2(1/p_i) \rceil$
Nyquist	$f_s \geq 2B$
Cuantificación	$\Delta = (V_{max} - V_{min})/L, L = 2^n$
Tasa de bits	$R = f_s \cdot n \cdot c$
Extensión de orden n	$H(S^n) = nH(S), L_n/n \rightarrow H(S)$
Markov: long. promedio	$L_M = \sum_i P(S_i)L_M(S_i)$
Markov: entropía	$H(S_M) \leq H(S)$
Distribución estacionaria	$\pi P = \pi, \sum \pi_i = 1$
Aritmética: intervalo	$newLow = low + rango \cdot Q_s, newHigh = low + rango \cdot R_s$
PPM-C: probabilidades	$P(ESC) = \frac{U}{total + U}, P(s) = \frac{freq(s)}{total + U}$

9. Preguntas teóricas cerradas (con respuesta breve)

1. **¿LZW antes o después de Huffman?** → Primero LZW, después Huffman: LZW saca redundancia de largo alcance (crea el diccionario), y sobre ese texto ya más "plano" Huffman asigna códigos cortos a lo frecuente.
2. **Fuente de 7 símbolos donde Huffman=Shannon=Fano** → cuando $p_i = r^{-l_i}$ exactamente para todos los símbolos (ej. 1/2, 1/4, 1/8, 1/16, 1/32, 1/64, 1/64): Kraft=1, H=L. Es el caso **ideal** de la codificación de fuente: si las probabilidades son potencias exactas de la base r , la longitud óptima $l_i = \log_r(1/p_i)$ ya da un número entero, sin nada para redondear. Por eso los tres métodos — que en general dan resultados distintos— coinciden acá: Shannon no tiene nada que redondear hacia arriba ($\lceil \cdot \rceil$ no cambia nada), Huffman arma un árbol óptimo que resulta tener exactamente esas longitudes, y Fano divide el alfabeto en mitades perfectamente equilibradas. Kraft=1 confirma que el árbol de prefijos queda completamente lleno (no sobra ningún código), y H=L es la definición misma de **código compacto** ($\eta = 100\%, R = 0\%$): no hay ningún margen de mejora teórica posible. Fuera de este caso especial (como en ABRACADABRA, donde las probabilidades no son potencias de 2), siempre queda algo de "pérdida por redondeo", y ahí aparece la desigualdad estricta $H \leq L < H + 1$.
3. **¿Para qué sirve la compansión?** → eficiencia en almacenamiento y transmisión, ahorro de costos (compresión + expansión combinadas).
4. **¿Instantáneo implica unívoco, o al revés?** → todo instantáneo es unívoco; no toda secuencia unívoca es instantánea. Huffman siempre produce código instantáneo.
5. **¿Los árboles de Huffman estático y dinámico coinciden siempre?** → No: el dinámico se adapta mientras procesa, el estático se arma con las frecuencias del mensaje completo ya conocidas.
6. **Peor canal ternario uniforme** → todas las probabilidades en 1/3, $C = 0$; emisor y receptor se tratan como independientes (el receptor "ya sabe" lo que va a llegar, no hay información nueva).
7. **¿Los métodos estáticos son de doble lectura? ¿Con o sin pérdida?** → Sí, doble lectura (primero cuentan frecuencias, después codifican); son sin pérdida.
8. **¿A mayor o menor redundancia se comprime más?** → A mayor redundancia, mayor margen de compresión (más patrones repetidos para explotar).
9. **Peor BSC** → $p = 0,5, C = 0$ (nunca se sabe si el bit recibido corresponde a un 0 o a un 1 enviado).
10. **¿Por qué un archivo comprimido tiene mayor entropía y menor redundancia que el original?** → porque la compresión elimina justamente los patrones predecibles; lo que queda es más "sorpresivo" en promedio (más incertidumbre por símbolo).

11. **BWT inversa** → reconstruir la cadena original a partir de la transformada, reordenando columnas iterativamente hasta encontrar la fila terminada en el símbolo de fin de cadena.

Nota de cobertura

Los parciales corregidos de 2023 usados como referencia para este apunte fueron resoluciones manuscritas de estudiantes de la misma batería que el examen modelo (4 variantes), más un Recuperatorio Tema 2 con canal binario asimétrico y decodificación de Huffman dinámico. Algunas de esas resoluciones contienen errores aritméticos (el caso más claro: una $I(A; B)$ negativa, matemáticamente imposible). En este apunte **todos los resultados numéricos fueron recalculados de forma independiente** —los de la sección 6, además, verificados con cálculo exacto por computadora (fracciones para Huffman/Shannon/Fano/aritmética, y una simulación completa con decodificación verificada para el Huffman dinámico)— por lo que no arrastran esos errores.